Guía de introducción y uso de Flet

Diseño de Aplicaciones Orientadas a Objetos (DAO) · UTN Facultad Regional Córdoba · 2026

Material de estudio y práctica · Python · Interfaces gráficas · Programación orientada a objetos

Objetivos de aprendizaje

- Comprender qué es Flet y sus aplicaciones.
- Crear interfaces gráficas con controles y contenedores.
- Programar eventos y manipular valores de los controles.
- Organizar una interfaz mediante clases y separar presentación, negocio y persistencia.

1. ¿Qué es Flet?

Flet es un framework que permite construir interfaces gráficas con Python utilizando controles como botones, campos de texto, listas, tablas y formularios. Permite desarrollar aplicaciones de escritorio, web y móviles sin escribir directamente HTML, CSS o JavaScript para cada pantalla.

En DAO, Flet se utiliza principalmente para construir la capa de presentación de aplicaciones orientadas a objetos. No reemplaza las reglas de negocio ni la persistencia.

Ejemplos de uso

- ABM de alumnos, productos o clientes.
- Formularios para registrar y consultar información.
- Aplicaciones administrativas con SQLite.
- Prototipos y herramientas internas.

2. Relación con los contenidos de DAO

Concepto	Aplicación con Flet
Clases y objetos	Clases que organizan formularios y controles
Encapsulamiento	Atributos y métodos para administrar una pantalla
Colaboración	La interfaz invoca objetos de negocio
Responsabilidades	Separación entre presentación, negocio y datos
Eventos	Métodos que responden a acciones del usuario
Pruebas	Lógica de negocio comprobable sin abrir la interfaz

3. Instalación y primer programa

Se recomienda utilizar una versión de Python compatible con la versión de Flet instalada y trabajar dentro de un entorno virtual. Verificar los requisitos de la documentación oficial antes de instalar.

```
python --version
python -m venv .venv
# Windows PowerShell:
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade "flet[all]"
flet doctor
```

Crear el archivo main.py:

```
import flet as ft

def main(page: ft.Page):
    page.title = "Mi primera aplicación"
    mensaje = ft.Text("Hola desde Flet")
    page.add(mensaje)

if __name__ == "__main__":
    ft.run(main)
```

Ejecutar con python main.py. Para desarrollo también puede utilizarse flet run main.py, según la instalación.

4. Estructura de una aplicación

- import flet as ft: importa la biblioteca.
- main(page): función de entrada que recibe la página.
- ft.Text(...): crea un control visual.
- page.add(...): agrega controles a la página.
- ft.run(main): inicia la aplicación.

5. Controles frecuentes

Control	Uso
ft.Text	Mostrar texto
ft.TextField	Ingresar datos
ft.Button	Ejecutar acciones
ft.Dropdown	Seleccionar opciones
ft.Checkbox	Marcar una opción
ft.Row / ft.Column	Distribución horizontal / vertical
ft.Container	Agrupar y dar formato
ft.DataTable	Mostrar datos tabulares

```
nombre = ft.TextField(label="Nombre")
tipo = ft.Dropdown(
    label="Tipo de documento",
    options=[
```

```
        ft.DropdownOption(key="DNI", text="DNI"),
        ft.DropdownOption(key="Pasaporte", text="Pasaporte"),
    ],
)
# Leer el contenido ingresado:
valor = nombre.value
```

6. Organización visual con Row y Column

Column ubica los controles de arriba hacia abajo; Row los distribuye horizontalmente. Se pueden anidar para construir formularios.

```
formulario = ft.Column(
    controls=[
        ft.Text("Datos personales"),
        ft.TextField(label="Nombre"),
        ft.TextField(label="Apellido"),
        ft.Row(controls=[
            ft.Button("Guardar"),
            ft.Button("Cancelar"),
        ]),
    ]
)
```

7. Eventos: responder a las acciones

El atributo on_click recibe una referencia a una función o método, no el resultado de ejecutarlo. El parámetro e representa el evento.

```
import flet as ft

def main(page: ft.Page):
    nombre = ft.TextField(label="Nombre")
    resultado = ft.Text()

    def saludar(e):
        resultado.value = f"Hola, {nombre.value}"

    page.add(nombre, ft.Button("Saludar", on_click=saludar), resultado)

ft.run(main)
```

En los manejadores de eventos habituales de Flet 1.0, los cambios de controles se sincronizan automáticamente al finalizar el evento. Consultar la documentación si se actualiza la interfaz fuera de ese contexto.

8. Organizar la interfaz con una clase

Para DAO es útil que una clase convencional de Python conserve referencias a los controles y concentre sus métodos de presentación. No es necesario heredar de UserControl, que pertenece a ejemplos antiguos.

```
import flet as ft
```

```
class SaludoApp:
    def __init__(self):
        self.nombre = ft.TextField(label="Nombre")
        self.resultado = ft.Text()

    def saludar(self, e):
        self.resultado.value = f"Hola, {self.nombre.value}"

    def build(self):
        return ft.Column(controls=[
            self.nombre,
            ft.Button("Saludar", on_click=self.saludar),
            self.resultado,
        ])

def main(page: ft.Page):
    app = SaludoApp()
    page.add(app.build())

ft.run(main)
```

Identificar en el código: clase, objeto, constructor, atributos, métodos, referencias y colaboración con los objetos de Flet.

9. Separación en capas

Presentación (Flet) → Negocio (validaciones y reglas) → Acceso a datos (SQLite). La interfaz recoge información y muestra resultados; no debería incluir consultas SQL ni reglas propias del dominio.

```
def agregar_alumno(self, e):
    mensaje = self.business.agregar_alumno(
        self.tipo_documento.value,
        self.numero_documento.value,
        self.apellido.value,
        self.nombre.value,
        self.fecha_nacimiento.value,
    )
    self.resultado.value = mensaje
```

10. Ejemplo práctico: registro de alumnos en memoria

El siguiente ejemplo no persiste datos: sirve para ejercitar controles, eventos, objetos y colecciones. Más adelante se podrá incorporar una BusinessLayer y SQLite.

```
import flet as ft

class AlumnosApp:
    def __init__(self):
        self.alumnos = []
        self.nombre = ft.TextField(label="Nombre")
        self.apellido = ft.TextField(label="Apellido")
        self.documento = ft.TextField(label="Documento")
        self.resultado = ft.Text()
        self.lista = ft.Column()

    def agregar(self, e):
        nombre = (self.nombre.value or "").strip()
```

```
        apellido = (self.apellido.value or "").strip()
        documento = (self.documento.value or "").strip()
        if not nombre or not apellido or not documento:
            self.resultado.value = "Complete todos los campos."
            return
        alumno = {"nombre": nombre, "apellido": apellido,
                  "documento": documento}
        self.alumnos.append(alumno)
        self.lista.controls.append(
            ft.Text(f"{apellido}, {nombre} - {documento}")
        )
        self.resultado.value = "Alumno agregado."
        self.nombre.value = ""
        self.apellido.value = ""
        self.documento.value = ""

    def build(self):
        return ft.Column(controls=[
            ft.Text("Registro de alumnos", size=24),
            self.nombre, self.apellido, self.documento,
            ft.Button("Agregar alumno", on_click=self.agregar),
            self.resultado,
            ft.Text("Alumnos registrados:"), self.lista,
        ], spacing=12, scroll=ft.ScrollMode.AUTO)

def main(page: ft.Page):
    page.title = "Registro de Alumnos"
    page.window.width = 600
    page.window.height = 650
    page.add(AlumnosApp().build())

if __name__ == "__main__":
    ft.run(main)
```

11. Actividades propuestas

Actividad 1. Ampliar el formulario

Agregar tipo de documento, correo y fecha de nacimiento; validar campos obligatorios.

Actividad 2. Mostrar y limpiar

Presentar alumnos registrados y agregar un botón para limpiar el formulario.

Actividad 3. Separar responsabilidades

Crear BusinessLayer y trasladar allí las reglas de registro y validación.

Actividad 4. Persistencia

Incorporar SQLite sin ejecutar SQL directamente desde los eventos.

12. Errores frecuentes

- Usar ft.UserControl en ejemplos actuales: preferir una clase convencional con build() para estos ejercicios.
- Confundir el control con su contenido: leer y modificar la propiedad .value.

- Escribir on_click=self.agregar(): esto ejecuta el método inmediatamente; usar on_click=self.agregar.
- Colocar reglas de negocio y SQL en el código de los botones.
- Copiar ejemplos de versiones anteriores sin comprobar los cambios de API.

13. Referencias oficiales

https://flet.dev/docs/

https://flet.dev/docs/controls/

https://flet.dev/docs/updates/migrate-to-1-0/

Nota: Flet evoluciona con rapidez. Verificar nombres de controles y compatibilidad de versiones al preparar el entorno de laboratorio.